

Neural Networks Assignment Report

Group 17

March 18, 2026

Part 1: The Bottleneck Challenge

Goal: Understand how a network compresses high-dimensional data into a "Latent Space."

1. The Task

A symmetric Fully Connected Autoencoder was trained on the standard MNIST dataset. The experiment focuses on understanding the impact of the bottleneck size of the autoencoders by testing two different models.

- **Model ($N = 2$):** A standard sequential autoencoder with a heavily compressed 2-dimensional latent space.
- **Model ($N = 32$):** A standard sequential autoencoder with a larger, 32-dimensional latent space.

2. Network architectures

- **Symmetric Autoencoder :** This is basically a standard Multi-Layer Perceptron (MLP) and the input image is a flattened vector of size $(28 \times 28, 1)$.

Encoder:

- **Input:** 784 neurons (Flattened 28×28 image).
- **Hidden Layer 1:** 128 neurons (ReLU activation) .
- **Hidden Layer 2:** 64 neurons (ReLU) .
- **Bottleneck:** N neurons (Linear) .

Decoder:

- **Hidden Layer 3:** 64 neurons (ReLU activation) .
- **Hidden Layer 4:** 128 neurons (ReLU) .
- **Output:** 784 neurons (Sigmoid activation) reshaped to 28×28 .

3. Training details

The models were trained under the following conditions:

- **Optimizer:** Adam
- **Loss Function:** Mean Squared Error (MSE) Loss
- **Epochs:** 30
- **Batch Size:** 128
- **Dataset:** Normal MNIST (training Set)

4. Analysis

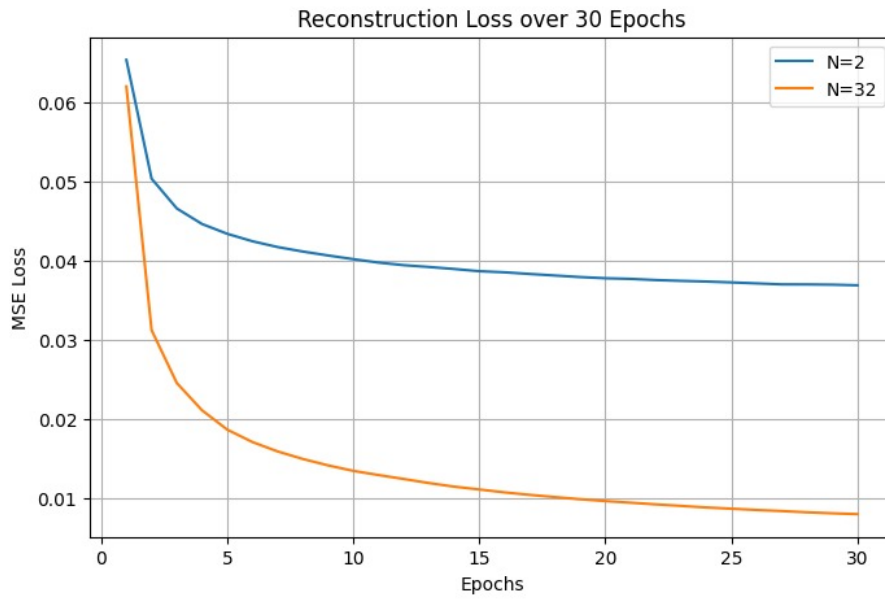


Figure 1: Reconstruction Loss over 30 epochs for $N = 2$ vs $N = 32$



Figure 2: Comparison of Original vs. Reconstructed images

Observations

There is a highly noticeable difference in visual quality. The FCNN model with $N = 2$ produces extremely blurry and generic reconstructions. Structurally similar digits bleed into one another, resulting in an ambiguous average shape. Conversely, the $N = 32$ model produces reconstructions that are much sharper and closely mirror the original input, preserving distinct features like slant, stroke thickness, and specific loops.

Why did the $N=2$ model struggle?

This difference is dictated by the capacity of the information bottleneck. The autoencoder attempts to compress a 784-dimensional input into an N -dimensional vector. When $N = 2$, the network is forced to achieve a massive compression ratio. It can only retain the most dominant, high-level global variations (e.g., overall digit roundness) and must discard all fine details.

Why did the $N=32$ model succeed?

When the bottleneck is increased to $N = 32$, the latent space becomes large enough to store much more than just the general outline of the digits. It has the capacity to capture and retain the specific, localized details of the original image. Because the decoder receives this richer set of encoded features, it doesn't have to guess how to rebuild the image. It can reliably map those learned details back to their precise pixel coordinates, which ultimately yields significantly sharper reconstructions and a much lower mean squared error (MSE) loss.

Part 2: Denoising & Feature Robustness

Goal: Force the network to learn useful features by corrupting the input.

Question 2.1: The Denoising Autoencoder (DAE)

Dataset: Tiny-ImageNet-10

The objective of this task is to train a convolutional autoencoder capable of reconstructing clean images from corrupted inputs. Noise is deliberately added to the images to force the model to learn robust representations of the data.

1. Data Corruption

Two types of noise can be applied to the images before feeding them to the encoder:

- **Gaussian Noise:** Random noise sampled from a normal distribution is added to pixel values.
- **Salt and Pepper Noise:** Random pixels are replaced with either minimum or maximum intensity values.

This forces the autoencoder to learn meaningful structural patterns instead of memorizing the raw pixel values.

2. Network Architecture

A Convolutional Denoising Autoencoder was implemented using PyTorch.

Encoder:

- Input Image: $64 \times 64 \times 3$
- Conv Layer: $3 \rightarrow 32$ channels, ReLU
- Max Pooling: $64 \rightarrow 32$
- Conv Layer: $32 \rightarrow 64$ channels, ReLU
- Max Pooling: $32 \rightarrow 16$
- Conv Layer: $64 \rightarrow 128$ channels, ReLU
- Max Pooling: $16 \rightarrow 8$

The encoder compresses the input image into a latent feature representation.

Decoder:

- Transposed Convolution: $128 \rightarrow 64$
- Transposed Convolution: $64 \rightarrow 32$
- Transposed Convolution: $32 \rightarrow 3$
- Sigmoid Activation

The decoder reconstructs the clean image from the encoded representation.

3. Training Details

The model was trained with the following settings:

- **Optimizer:** Adam
- **Loss Function:** Mean Squared Error (MSE)
- **Epochs:** 10

The training objective is to minimize the reconstruction loss between the predicted clean image and the ground truth clean image.

5. Reconstruction Results

To visually evaluate the performance of the denoising autoencoder, a reconstruction comparison was generated consisting of:

1. Original Clean Image
2. Noisy Input Image
3. Model Denoised Output

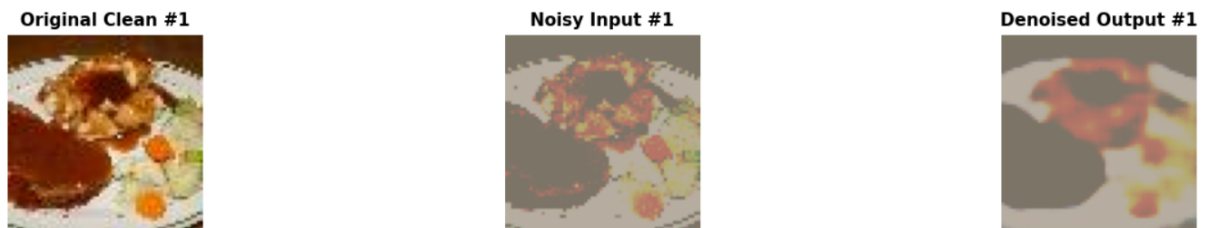


Figure 3: Reconstruction Trio: Original Image, Noisy Input, and Model Output

6. Analysis

The results show that the denoising autoencoder successfully removes a significant portion of the noise from the corrupted input images. Although some fine details are lost during reconstruction, the model preserves the overall structure and main features of the objects.

The blurring effect occurs because the Mean Squared Error loss encourages the network to predict the average of possible pixel values, which smooths out high-frequency details.

7. Conclusion

The experiment demonstrates that denoising autoencoders can effectively learn robust feature representations by reconstructing clean images from noisy inputs. This technique helps neural networks focus on meaningful structures rather than noise in the data.

Part 3: Latent Space Visualization and Anomaly Detection

3.1 Latent Space Visualization (2D Manifold)

Goal

The goal of this experiment is to visualize how the autoencoder organizes handwritten digit images in its latent space. By restricting the latent dimension to two ($N = 2$), we can project the encoded representations of MNIST images into a two-dimensional manifold and observe how similar digits cluster together.

Methodology

A trained autoencoder was used where the encoder compresses each input image into a two-dimensional latent representation. The entire MNIST test dataset was passed through the encoder to obtain the latent coordinates for each image. Each encoded sample corresponds to a point in the two-dimensional latent space. These points were plotted using a scatter plot where each point is colored according to its true digit label. This visualization allows us to observe how the model organizes similar digit structures in the learned feature space.

Architecture

Encoder

Input: 784 neurons(Flattened 28×28 MNIST image)

Hidden layer 1: 128 neurons(RELU)

Hidden layer 2: 64 neurons(RELU)

Latent layer: 2 neurons(linear)

Decoder

Hidden layer 3: 64 neurons(RELU)

Hidden layer 4: 128 neurons(RELU)

Output: 784 neurons(sigmoid)

The encoder compresses the image into a 2-dimensional representation, while the decoder reconstructs the original image from this compact latent representation.

Latent Space Representation

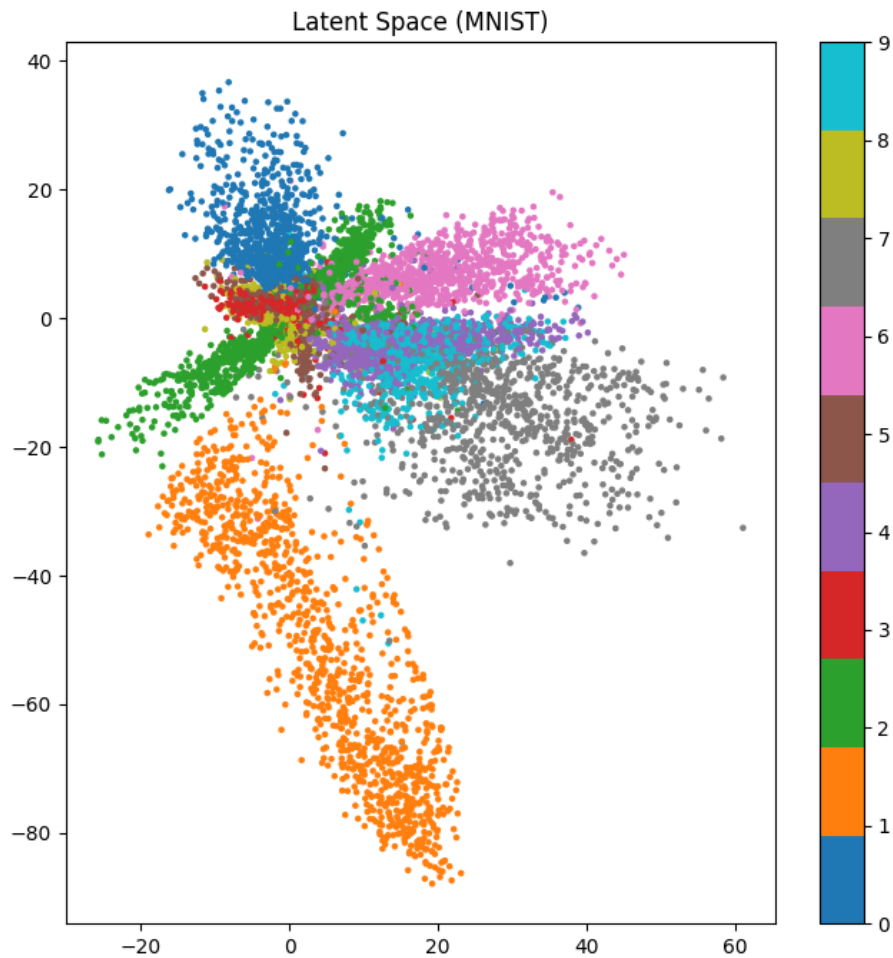


Figure 4: Two-dimensional latent space representation of MNIST digits produced by the encoder.

Observation

- Digits with similar visual structures tend to form nearby clusters (like 1 and 7).
- Some digits overlap in the latent space due to similarities in their shapes.
- Distinct digits tend to form compact and clearly separated clusters.

Interpretation

The latent space visualization demonstrates that the autoencoder learns meaningful representations of handwritten digits. Instead of storing raw pixel information, the encoder captures structural patterns of digits and maps similar images to nearby regions in the latent space.

This indicates that the autoencoder performs effective dimensionality reduction while preserving important relationships between samples.

Conclusion

The results show that the autoencoder successfully compresses high-dimensional image data into a meaningful two-dimensional representation. The formation of clusters corresponding to different digits indicates that the encoder learns structured feature representations that capture similarities between images.

3.2 Anomaly Detection Using Reconstruction Error

Goal

The objective of this experiment is to evaluate whether an autoencoder can be used for anomaly detection. The idea is that an autoencoder trained on a subset of classes should reconstruct those classes well while producing higher reconstruction errors for unseen classes.

Methodology

The autoencoder was trained using images from **9 out of the 10 encoded classes** in the Tiny-ImageNet-10 dataset. One class was intentionally excluded from the training set and treated as an unseen or anomalous class. After training, both seen and unseen class images were passed through the autoencoder. For each image, the reconstruction error was computed using Mean Squared Error (MSE) between the original input and the reconstructed output.

Architecture

This model uses an Autoencoder trained on 9 classes of Tiny-ImageNet-10 to learn normal image patterns. Images from the unseen class are expected to produce higher reconstruction errors.

Encoder

Input: 64x64x3

Conv2D: 32 filters, 3x3 kernels, RELU

Maxpooling2D: 2x2 pool size

Conv2D: 64 filters, 3x3 kernels, RELU

Maxpooling2D: 2x2 pool size

Conv2D: 128 filters, 3x3 kernels, RELU

Latent representation: flattened feature vector

Decoder

Conv2DTranspose: 128 filters, 3x3, RELU

Upsampling2D: 2x2

Conv2DTranspose: 64 filters, 3x3, RELU

Upsampling2D: 2x2

Conv2DTranspose: 32 filters, 3x3, RELU

output: 64x64x3 image(sigmoid activation)

Reconstruction Error Comparison

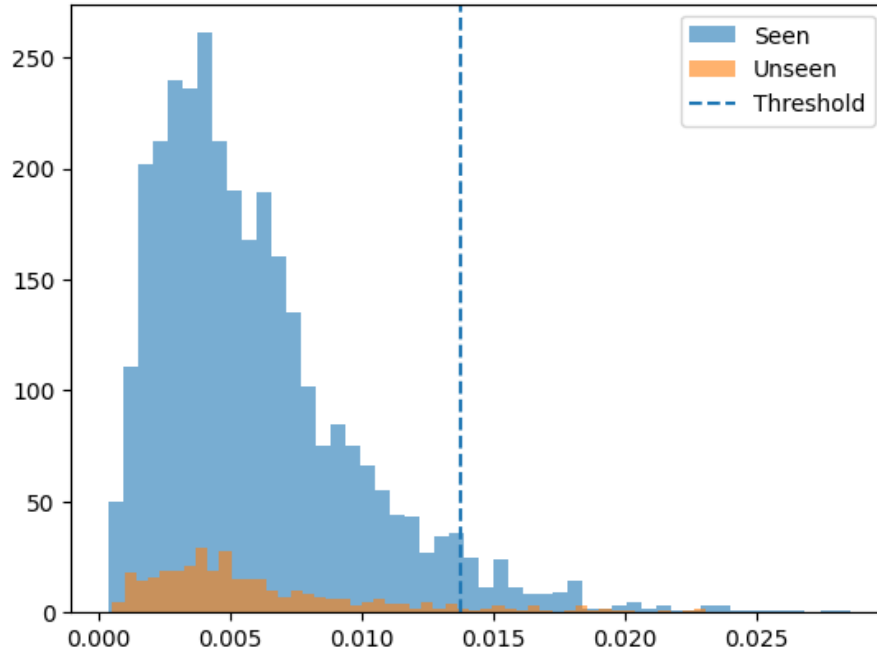


Figure 5: Comparison of reconstruction errors for images belonging to seen classes and the unseen class.

Observations

- Images from seen classes generally have lower reconstruction errors.
- Images from the unseen class produce higher reconstruction errors.
- The difference in errors helps distinguish normal samples from anomalous ones.

Interpretation

Since the autoencoder is trained only on specific classes, it learns the underlying patterns of those classes. When it encounters images from an unseen class, it cannot reconstruct them accurately, resulting in higher reconstruction errors. This difference in reconstruction performance allows the model to identify anomalous samples.

Conclusion

The experiment demonstrates that reconstruction error can effectively be used as an anomaly detection metric. Images from unseen classes consistently produce higher reconstruction errors, allowing the autoencoder to distinguish between normal and anomalous data samples.

Part 3: Ablation Study:

Tanh vs Sigmoid

The sigmoid activation function performed better because the reconstructed pixel values are constrained to the same range as the normalized image data ($[0,1]$). This allows the network to learn a more accurate reconstruction compared to tanh, whose output range is $[-1,1]$.